# 1. Consultas Básicas (Proyecciones y Alias)

Estas consultas sirven para extraer columnas específicas y renombrarlas para que el informe sea más legible.

Listar todos los productos con su marca y modelo:
SELECT Marca, Modelo FROM Producto;

Mostrar el nombre de las tiendas y el código de país, usando alias para las columnas:
SELECT Nombre AS Nombre_Tienda, CodPais AS Pais_Origen FROM Tienda;

Obtener los nombres completos de los clientes en una sola columna (Concatenación):
SELECT Nombre || ' ' || PrApellido || ' ' || SgApellido AS Cliente_Completo FROM Cliente;

# 2. Consultas con Filtros (WHERE)

Aquí empezamos a segmentar la información según condiciones específicas.

Productos que cuestan más de 400€ (Precio de venta):
SELECT * FROM Producto WHERE PrecioVenta > 400;

Clientes cuyo primer apellido sea 'Prada':
SELECT * FROM Cliente WHERE PrApellido = 'Prada';

Fabricantes fundados entre 1930 y 1980:
SELECT Nombre, AnyoFundacion FROM Fabricante
WHERE AnyoFundacion BETWEEN 1930 AND 1980;

Pedidos realizados en una fecha específica:

SELECT * FROM Pedido WHERE FechaPed = DATE '2022-03-22';

# 3. Uso de Operadores Lógicos y de Conjuntos (LIKE, IN, IS NULL)

Ideales para búsquedas más flexibles.

Productos de la marca Apple o Samsung:
SELECT * FROM Producto WHERE Marca IN ('Apple', 'Samsung');

Clientes que no tienen registrado su segundo apellido:
-- En tu DML hay algunos con cadena vacía '', pero si fueran NULL:
SELECT * FROM Cliente WHERE SgApellido IS NULL OR SgApellido = '';

Productos cuyo modelo contenga el número '1':
SELECT * FROM Producto WHERE Modelo LIKE '%1%';

# 4. Ordenación y Cálculos Simples

Listar productos ordenados por precio de venta (de mayor a menor):
SELECT Marca, Modelo, PrecioVenta
FROM Producto
ORDER BY PrecioVenta DESC;

Calcular el margen de beneficio por producto:
SELECT Marca, Modelo, (PrecioVenta - PrecioCoste) AS Beneficio
FROM Producto;

Mostrar los 3 productos más baratos:

```
SELECT * FROM Producto ORDER BY PrecioVenta ASC FETCH
FIRST 3 ROWS ONLY;
```

## 5. Funciones de Agregado (Resúmenes)

Útiles para obtener estadísticas rápidas de la tienda.

¿Cuántos pedidos se han realizado en total?

```
SELECT COUNT(*) AS Total_Pedidos
FROM Pedido;
```

¿Cuál es el precio medio de los productos de la tienda?

```
SELECT AVG(PrecioVenta) AS Precio_Medio
FROM Producto;
```

Precio máximo y mínimo de los productos en stock:

```
SELECT MAX(PrecioVenta) AS Caro, MIN(PrecioVenta) AS Barato
FROM Producto;
```

## 1. Consultas con INNER JOIN (Cruce de datos)

El INNER JOIN solo devuelve filas donde hay una coincidencia en ambas tablas. Es el más común.

Ver el nombre del cliente y el país al que pertenece:
En lugar de ver solo el CodPais (un número), traemos el Nombre de la tabla Pais.

```
SELECT C.Nombre, C.PrApellido, P.Nombre AS Pais
FROM Cliente C
INNER JOIN Pais P ON C.CodPais = P.CodPais;
```

Listar productos junto con el nombre de su fabricante:

```
SELECT Pro.Marca, Pro.Modelo, Fab.Nombre AS Fabricante
FROM Producto Pro
JOIN Fabricante Fab ON Pro.CodFab = Fab.CodFabricante;
```

Detalle de pedidos: ¿Quién compró qué y en qué color?
Aquí unimos 4 tablas para obtener un informe legible.

```
SELECT Pe.CodPedido, Cl.Nombre AS Cliente, Pr.Modelo AS Producto,
Co.Color
FROM Pedido Pe
JOIN Cliente Cl ON Pe.CodCliente = Cl.CodCliente
JOIN Producto Pr ON Pe.CodProducto = Pr.CodProducto
JOIN Color Co   ON Pe.CodColor = Co.CodColor;
```

# 2. Consultas con Agrupación (GROUP BY)

Estas son fundamentales para analítica y reportes de ventas.

¿Cuántos clientes hay por cada país?

```
SELECT P.Nombre, COUNT(C.CodCliente) AS Numero_Clientes
FROM Pais P
JOIN Cliente C ON P.CodPais = C.CodPais
GROUP BY P.Nombre;
```

Dinero total recaudado por cada tienda:
Multiplicamos la cantidad por el precio de venta del producto.

```
SELECT T.Nombre AS Tienda, SUM(Pe.Cantidad * Pr.PrecioVenta) AS
Total_Ventas
FROM Pedido Pe
JOIN Tienda T ON Pe.CodTienda = T.CodTienda
JOIN Producto Pr ON Pe.CodProducto = Pr.CodProducto
GROUP BY T.Nombre;
```

## 3. Consultas con Filtros de Grupo (HAVING)

El WHERE filtra filas; el HAVING filtra los resultados después de haber agrupado.
Marcas que tienen más de 2 modelos registrados:

```
SELECT Marca, COUNT(*) AS Modelos_Disponibles
FROM Producto
GROUP BY Marca
HAVING COUNT(*) > 2;
```

## 4. El "Bonus" de Subconsultas

A veces necesitas una consulta dentro de otra para responder preguntas complejas.
Mostrar los productos que son más caros que el precio medio de la tienda:

```
SELECT Marca, Modelo, PrecioVenta
FROM Producto
WHERE PrecioVenta > (SELECT AVG(PrecioVenta) FROM Producto);
```

El UPDATE cambia valores en columnas específicas de filas que cumplan una condición.
Actualizar el precio de un producto específico:
Supongamos que el iPhone 11 (CodProducto 1) ha subido de precio.

```
UPDATE Producto
SET PrecioVenta = 699.99
WHERE CodProducto = 1;
```

Aplicar un descuento masivo (Rebajas):
Vamos a bajar un 10% el precio de venta de todos los productos de la marca 'Samsung'.

```
UPDATE Producto
SET PrecioVenta = PrecioVenta * 0.90
WHERE Marca LIKE 'Samsung%';
```

Corregir datos de un cliente:
Daniel Romero (CodCliente 2) se ha mudado y ahora pertenece al país con código 2.

```
UPDATE Cliente
SET CodPais = 2
WHERE CodCliente = 2;
```

## 2. Sentencias DELETE (Borrar datos)

El DELETE elimina filas completas. Ten en cuenta que en tu base de datos tienes ON DELETE CASCADE, lo que significa que si borras un registro "padre", se borrarán sus "hijos" automáticamente.

Eliminar un pedido específico:

```
DELETE FROM Pedido
WHERE CodPedido = 8 AND CodCliente = 4;
```

Eliminar todos los productos de un fabricante:
Si eliminamos al fabricante Apple (CodFab 1), debido al CASCADE, se borrarán sus productos y los pedidos asociados a esos productos.

```
DELETE FROM Fabricante
WHERE CodFabricante = 1;
```

Borrar clientes que no han hecho pedidos en 2021:
Usando una subconsulta para identificar a quién borrar.

```
DELETE FROM Cliente
WHERE CodCliente NOT IN (
   SELECT CodCliente FROM Pedido WHERE FechaPed BETWEEN
DATE '2021-01-01' AND DATE '2021-12-31');
```

# 1. Subconsultas en el WHERE (Filtrado dinámico)

Estas sirven para comparar un valor individual con un resultado calculado sobre la marcha.

Productos con precio superior a la media de la tienda:

```
SELECT Marca, Modelo, PrecioVenta
FROM Producto
WHERE PrecioVenta > (SELECT AVG(PrecioVenta) FROM Producto);
```

Clientes que han realizado pedidos en una tienda específica (ej. 'Sworten'):

```
SELECT Nombre, PrApellido
FROM Cliente
WHERE CodCliente IN (
   SELECT CodCliente
   FROM Pedido
   WHERE CodTienda = (SELECT CodTienda FROM Tienda WHERE
Nombre = 'Sworten'));
```

# 2. Subconsultas con ALL, ANY y SOME

Se usan cuando la subconsulta devuelve una lista de valores.

Producto que es más caro que TODOS los productos de la marca Samsung:

```
SELECT Marca, Modelo, PrecioVenta
FROM Producto
WHERE PrecioVenta > ALL (
   SELECT PrecioVenta FROM Producto WHERE Marca LIKE
'Samsung%');
```

Fabricantes que tienen al menos un producto con precio de coste inferior a 200€:

```
SELECT Nombre
FROM Fabricante
WHERE CodFabricante = ANY (
```

```
    SELECT CodFab FROM Producto WHERE PrecioCoste < 200);
```

## 3. Subconsultas Correlacionadas

Son aquellas donde la consulta interna hace referencia a una columna de la consulta externa. Se ejecutan fila por fila.

Listar productos cuyo precio de venta sea el máximo dentro de su propia marca:

```
SELECT P1.Marca, P1.Modelo, P1.PrecioVenta
FROM Producto P1
WHERE P1.PrecioVenta = (
    SELECT MAX(P2.PrecioVenta)
    FROM Producto P2
    WHERE P2.Marca = P1.Marca);
```

## 4. Subconsultas en el FROM (Tablas derivadas)

A veces es más fácil tratar el resultado de una consulta como si fuera una tabla temporal para luego hacer un JOIN con ella.

Obtener el gasto total por cliente comparándolo con el gasto medio general:

```
SELECT T1.CodCliente, T1.GastoTotal
FROM (
    -- Esta es nuestra "tabla temporal"
    SELECT CodCliente, SUM(Cantidad * PrecioVenta) AS GastoTotal
    FROM Pedido pe
    JOIN Producto pr ON pe.CodProducto = pr.CodProducto
    GROUP BY CodCliente
) T1
WHERE T1.GastoTotal > (
```

```
   SELECT AVG(Cantidad * PrecioVenta) FROM Pedido pe JOIN
Producto pr ON pe.CodProducto = pr.CodProducto);
```

## 5. Uso de EXISTS y NOT EXISTS

Es la forma más eficiente de comprobar si existe un registro relacionado sin necesidad de traer todos los datos.

Paises que NO tienen ninguna tienda registrada:

```
SELECT Nombre
FROM Pais P
WHERE NOT EXISTS (
   SELECT 1 FROM Tienda T WHERE T.CodPais = P.CodPais
);
```

## 1. Consultas de Selección (DQL)

Son las herramientas que usas para leer datos de la base de datos.

Proyección y Alias: Usamos SELECT para elegir columnas específicas. Con AS les damos un nombre más amigable (ej. llamar "Local" a la columna "Tienda").

Concatenación: Con el operador || podemos pegar textos, como unir el Nombre y los Apellidos de un cliente en una sola frase.

Filtros de Fila (WHERE): Es el corazón de las consultas. Permite filtrar por números (> 100), por textos exactos (= 'Apple') o por rangos usando BETWEEN.

Búsquedas Flexibles: El operador LIKE con el comodín % sirve para buscar palabras que contengan ciertas letras (ej. modelos que tengan un "11"). El operador IN sirve para pasar una lista de valores permitidos.

Ordenación: Con ORDER BY decides si los datos aparecen de menor a mayor (ASC) o de mayor a menor (DESC).

## 2. Relaciones entre Tablas (JOINs)

Como tu base de datos tiene la información repartida (Países, Clientes, Productos), el JOIN es el pegamento.

Inner Join: Es la unión estándar. Cruza dos tablas por su clave común (ej. el CodPais que está en la tabla Cliente y en la tabla Pais). Si un cliente no tiene país o un país no tiene clientes, no aparecen en el resultado.

Joins Múltiples: Puedes encadenar tantos como quieras. Por ejemplo, para saber de qué país es el fabricante del producto que compró un cliente, tendrías que unir 5 tablas seguidas.

## 3. Funciones de Grupo y Agregación

Se usan para obtener totales o estadísticas en lugar de filas individuales.

Funciones Básicas: COUNT para contar registros, SUM para sumar valores monetarios, AVG para sacar medias de precios, y MAX/MIN para encontrar los extremos.

Agrupamiento (GROUP BY): Divide los datos en grupos antes de calcular. Por ejemplo: "Suma las ventas agrupadas por el nombre de la tienda".

Filtro de Grupos (HAVING): Es un filtro que se aplica después de agrupar. Si quieres ver solo las tiendas que han vendido más de 500€ en total, usas HAVING SUM(...) > 500.

## 4. Subconsultas (Consultas Anidadas)

Es meter una consulta dentro de otra para resolver preguntas lógicas complejas.

Escalares: La consulta interna devuelve un solo dato (como el precio medio) y la externa lo usa para comparar.

De Lista: La consulta interna devuelve una columna de datos y usamos IN o ANY para ver si un valor externo está en esa lista.

Correlacionadas: Son consultas "inteligentes" donde la parte de adentro sabe lo que está pasando afuera (ej. buscar el producto más caro de cada marca específica).

Existenciales (EXISTS): No devuelven datos, solo dicen "Sí" o "No". Sirven para saber si un registro tiene hijos en otra tabla (ej. países que tienen al menos una tienda).

## 5. Modificación de Datos (DML)

Aquí es donde la base de datos cambia físicamente.

UPDATE: Modifica valores de filas que ya existen. Puedes cambiar un solo dato o miles a la vez (como subir el precio a todos los iPhone un 5%).

DELETE: Borra filas completas. Gracias a tus restricciones de CASCADE, si borras un Fabricante, el sistema limpia automáticamente todos sus productos y pedidos asociados.

Control de Transacciones: En sistemas como Oracle, después de un Update o Delete, debes usar COMMIT para confirmar los cambios o ROLLBACK para arrepentirte y volver atrás.

# RESUMEN

**2. Consultas y Comandos de Ejemplo (DML/Básicos)**

Aunque el PDF se centra en el diseño físico (DDL), menciona ejemplos de consultas básicas:

- **Selección simple:** SELECT * FROM Profesores WHERE Nombre='Javier';.
- **Selección con condiciones:** SELECT * FROM Asignatura WHERE CodAsig > 2 AND CodAsig < 5;.
- **Selección de campos específicos:** SELECT Nombre FROM Asignatura WHERE CodAsig = 1;.

**3. Comandos de Definición de Datos (DDL)**

**Creación (CREATE)**

- **Tablas:** Se usa CREATE TABLE definiendo columnas, tipos de datos y restricciones.
    - Ejemplo: CREATE TABLE Profesor (CodProf number(14), ..., CONSTRAINT pk_Profesor PRIMARY KEY(CodProf));.
- **Otros objetos:** El documento menciona que se pueden crear: USER, VIEW, TABLESPACE, DATABASE, INDEX, SYNONYM, TYPE, FUNCTION, PROCEDURE, ROLE, PROFILE.

**Modificación (ALTER)**

Se utiliza para cambiar objetos ya existentes:

- **Añadir columna:** ALTER TABLE Ediccion ADD numTiradas Number(5);.
- **Modificar columna:** ALTER TABLE Ediccion MODIFY numTiradas Number(7) NOT NULL;.
- **Eliminar columna:** ALTER TABLE Ediccion DROP NumTiradas;.
- **Gestionar restricciones:** Permite añadir (ADD CONSTRAINT), modificar (MODIFY CONSTRAINT), eliminar (DROP CONSTRAINT) o desactivar/activar (DISABLE/ENABLE) restricciones.

#### Eliminación y Otros

- **DROP TABLE:** Elimina una tabla. Se recomienda usar CASCADE CONSTRAINTS si tiene referencias y PURGE para liberar espacio inmediatamente.
- **RENAME:** Cambia el nombre de una tabla u objeto (ej. RENAME Ediccion TO Periodico).
- **TRUNCATE:** Vacía todos los registros de una tabla sin eliminar su estructura.

### 4. Restricciones y Subconsultas Lógicas

El documento detalla cómo aplicar lógica a las definiciones y consultas:

- **Integridad referencial:** Uso de FOREIGN KEY con opciones como ON DELETE CASCADE o ON DELETE SET NULL.
- **Validación (CHECK):** Permite restringir valores mediante operadores lógicos y relacionales (IN, BETWEEN, AND, OR).
    - Ejemplo: CONSTRAINT ck_genero CHECK (Genero IN ('Narrativo', 'Lírico')).
- **Operadores relacionales:** =, !=, <>, >, <, >=, <=.
- **Operadores lógicos:** NOT, OR, AND.

**Para ver todas las restricciones de una tabla específica:**
SQL
SELECT A.CONSTRAINT_NAME, A.STATUS, A.DELETE_RULE, A.CONSTRAINT_TYPE

FROM ALL_CONSTRAINTS A

WHERE TABLE_NAME = 'COMPRA';

- 

**Para ver condiciones específicas (como las de tipo CHECK):**
SQL
SELECT CONSTRAINT_NAME, SEARCH_CONDITION

FROM ALL_CONSTRAINTS

WHERE TABLE_NAME = 'LINEACOMPRA' AND CONSTRAINT_NAME = 'CK_IVA';

- 
- **Consulta combinada para múltiples tablas:** El PDF muestra cómo listar el estado y tipo de restricción de las tablas `ESTUDIANTE`, `PROFESOR`, `ASIGNATURA` y `MATRICULA` de una sola vez usando la cláusula `IN`.

#### 3. Comandos de Creación de Estructuras (DDL)

El PDF detalla la creación de un modelo relacional completo. Aunque no son "consultas" de extracción, son las sentencias ejecutadas:

- **Tabla PROFESOR:** Definición de clave primaria (`PK_PROFESOR`).
- **Tabla ASIGNATURA:** Incluye una clave foránea que referencia a Profesor (`FK_ASIG_PROF`).
- **Tabla ESTUDIANTE:** Definición de datos personales.
- **Tabla MATRICULA:** Es la más compleja, ya que utiliza una **clave primaria compuesta** (DNI + CodAsig) y dos claves foráneas:
  - `FK_MATRIC_ESTUD` (referencia a Estudiante).
  - `FK_MATRIC_ASIG` (referencia a Asignatura).

#### 4. Gestión de Restricciones (Constraints)

Se presentan comandos para manipular las reglas de la base de datos sin borrar las tablas:

- **Desactivar una restricción:** `ALTER TABLE Ediccion DISABLE CONSTRAINT ck_numtiradas CASCADE;`
- **Activar una restricción:** `ALTER TABLE Ediccion ENABLE CONSTRAINT ck_numtiradas CASCADE;`
- **Uso de CASCADE:** Se enfatiza que el uso de `CASCADE` permite que la acción afecte a todas las restricciones que dependan de la principal.

#### 5. Comandos de Borrado y Limpieza

Para reiniciar el entorno de prácticas, el documento lista:

**Borrado masivo con integridad:**
SQL
DROP TABLE ASIGNATURA CASCADE CONSTRAINTS PURGE;

DROP TABLE ESTUDIANTE CASCADE CONSTRAINTS PURGE;

DROP TABLE MATRICULA CASCADE CONSTRAINTS PURGE;

- DROP TABLE PROFESOR CASCADE CONSTRAINTS PURGE;

#### 1. Estructura Básica de una Consulta (`SELECT`)

El documento establece el orden lógico de las cláusulas:

- **SELECT**: Columnas a mostrar (uso de `DISTINCT` para no repetir).
- **FROM**: Tablas de origen.
- **WHERE**: Condiciones de filtrado.
- **GROUP BY**: Agrupamiento de filas.
- **HAVING**: Condiciones sobre los grupos.
- **ORDER BY**: Ordenación de resultados (`ASC`|`DESC`).

**2. Operadores y Expresiones en Consultas**

- **Operadores de Comparación**: `=`, `<>`, `!=`, `<`, `>`, `<=`, `>=`.
- **Operadores Lógicos**: `AND`, `OR`, `NOT`.
- **Operadores Especiales**:
  - `IN / NOT IN`: Para listas de valores.
  - `BETWEEN / NOT BETWEEN`: Para rangos.
  - `IS NULL / IS NOT NULL`: Para valores inexistentes.
  - `LIKE`: Búsqueda de patrones con comodines (`%` para varios caracteres, _ para uno solo).

**3. Funciones de Agregado y de Fila**

- **Funciones de grupo**: `SUM`, `AVG`, `MAX`, `MIN`, `COUNT(*)`.
- **Funciones de cadena**: `UPPER`, `LOWER`, `CONCAT`, `SUBSTR`, `LENGTH`.
- **Funciones numéricas**: `ROUND`, `TRUNC`, `MOD`.
- **Funciones de fecha**: `SYSDATE`, `ADD_MONTHS`, `MONTHS_BETWEEN`.
- **Función de conversión/nulos**: `NVL(valor, sustituto)` para manejar nulos.

**4. Consultas Multitabla (Joins)**

El PDF explica cómo combinar datos de diferentes tablas:

- **Inner Join (Equijoin)**: Solo registros con coincidencia en ambas tablas.
- **Natural Join**: Unión automática por columnas con el mismo nombre.
- **Outer Joins**:
  - `LEFT OUTER JOIN`: Todos los de la izquierda + coincidencias.
  - `RIGHT OUTER JOIN`: Todos los de la derecha + coincidencias.
  - `FULL OUTER JOIN`: Todos los registros de ambas.
- **Self-Join**: Una tabla se une consigo misma (usando alias).

**5. Subconsultas**

Se detallan consultas anidadas dentro de otras:

- **Subconsultas de una sola fila**: Devuelven un único valor (usadas con `=`, `>`, etc.).
- **Subconsultas de múltiples filas**: Devuelven una lista (usadas con `IN`, `ANY`, `ALL`).
- **Subconsultas correlacionadas**: La consulta interna hace referencia a columnas de la consulta externa.
- **Operador EXISTS**: Para comprobar la existencia de registros que cumplan una condición.

**6. Operadores de Conjuntos**

Permiten combinar los resultados de dos o más consultas `SELECT` independientes:

- **UNION / UNION ALL**: Combina resultados (eliminando o manteniendo duplicados).
- **INTERSECT**: Registros comunes a ambas consultas.
- **MINUS**: Registros de la primera consulta que no están en la segunda.

**7. Vistas (Views)**

Consultas almacenadas como objetos lógicos:

- **Creación**: `CREATE VIEW nombre_vista AS SELECT...`.
- **Vistas Actualizables**: Permiten hacer `INSERT` o `UPDATE` sobre la vista (con la opción `WITH CHECK OPTION`).
- **Vistas Materializadas**: Almacenan físicamente el resultado de la consulta para mejorar el rendimiento (`CREATE MATERIALIZED VIEW`).

**8. Otros Comandos Relacionados**

- **Inserción mediante consultas**: `INSERT INTO tabla SELECT...` (copiar datos de una tabla a otra).

- **Borrado selectivo**: `DELETE FROM tabla WHERE columna = (SELECT...)`.